traces. This means you need a distributed tracing backend and to point Tracetest to it or, alternatively, directly send spans to Tracetest, using an OpenTelemetry collector, for example. You can use it to define tests and assertions against services along a request path and define assertions against both the response and trace data. In addition, you can run the tests manually or via CI build jobs. Figure 9.8 shows Tracetest test results, rendering the requests in a waterfall manner.

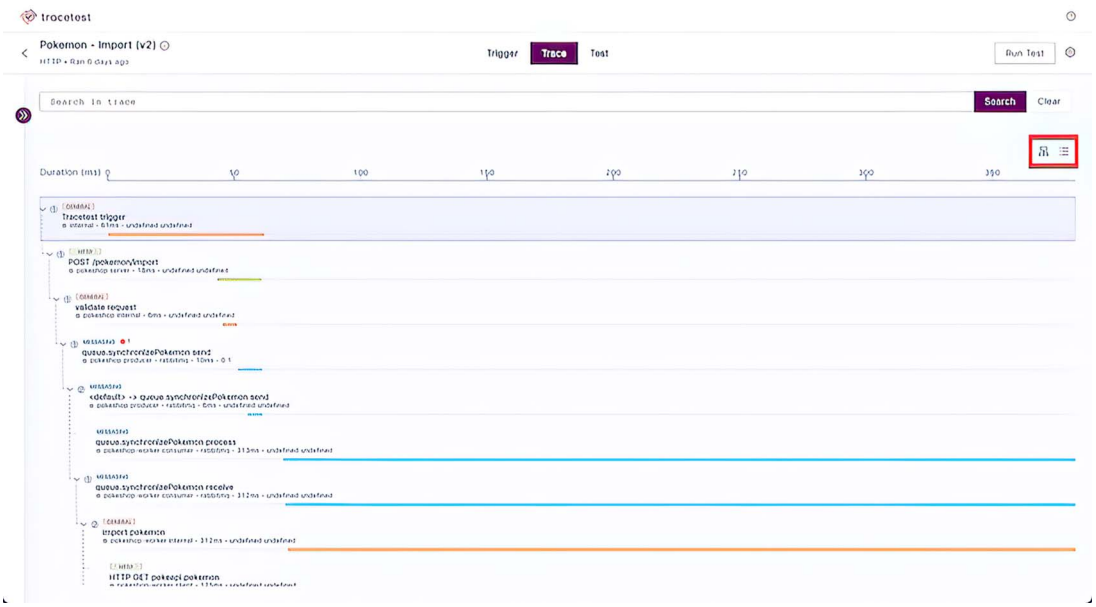


Figure 9.8 Screenshot of the Tracetest UI (results view)

You can install Tracetest (http://mng.bz/nWG5) via Docker Compose or a Helm chart and a number of example integrations, such as with AWS X-Ray (http://mng.bz/vnEp) or Grafana Tempo (http://mng.bz/4DwR).

Rookout

Rookout (https://www.rookout.com/) calls itself the "developer-first observability tool." It captures live debugging data across your stack, supporting Java, .NET, Node.js, Python, Ruby, and Go. Rookout allows you to live debug in a range of environments as well as view traces alongside available breakpoints.

Autometrics

Autometrics (https://autometrics.dev/) is built on top of Prometheus and OpenTelemetry client libraries. It makes it easy to instrument functions in your code (currently for Rust, Go, Python, Typescript, and C#) with common metrics, including request rate, error rate, and latency, and generates PromQL queries for you.